

# PV Energy Suite voor Home Assistant

Zonnepanelen voorspellen, slim verbruik plannen, negatieve prijzen beheren en fasebelasting bewaken - een complete gids voor PV Grid Forecast, Self-Consumption Planner, Phase Peak Guard en de PV-schakelblueprint.



## Waar deze suite voor bedoeld is

PV Energy Suite brengt vier losse functies samen zonder ze onnodig aan elkaar vast te lassen. Alleen de PV-blueprint mag de PV-schakelaar bedienen. Forecast, planner en Phase Guard analyseren en adviseren.

Gebaseerd op de actuele repository en README, opnieuw gecontroleerd op 5 oktober 2026. De README is sinds de eerdere controle daadwerkelijk gewijzigd; de nieuwe quick-start en apparaatconfiguratie zijn in deze gids verwerkt.

# 1. De README is veranderd - en dat maakt de installatie duidelijker

Ja: de README is sinds mijn eerdere controle aangepast. De inhoudelijke architectuur is hetzelfde gebleven, maar de instaproute is duidelijker geworden. De huidige README opent nu met een expliciet Begin hier-gedeelte en adviseert voor een eerste test PV Grid Forecast met de geïntegreerde planner.

De belangrijkste verduidelijkingen zijn:

- werk vanuit je bestaande AppDaemon apps.yaml; overschrijf dat bestand niet;
- voor de geïntegreerde planner kopieer je pv\_grid\_forecast.py én pv\_self\_consumption.py en zet je planner\_enabled: true;
- kies bewust tussen vaste YAML-waarden en Home Assistant-helpers voor apparaatprofielen;
- de README legt nu veel explicieter uit welke bronentiteiten je zelf moet leveren en welke MQTT-entiteiten de suite juist zelf aanmaakt;
- de waarschuwing om de geïntegreerde en standalone planner nooit tegelijk te draaien staat prominenter en concreter uitgelegd.

## Belangrijk voor dit artikel

De installatiehoofdstukken hieronder volgen deze nieuwere README-structuur, niet de oudere versie.

## Inhoud van deze gids

1. Wat de suite is · 2. De vier onderdelen · 3. Databronnen · 4. PV-schakeling · 5. PV Grid Forecast · 6. Self-Consumption Planner · 7. Geïntegreerd of standalone · 8. Phase Peak Guard · 9. Installatie · 10. Helpers en dashboards · 11. Voorbeeld van een dag · 12. Beperkingen, veiligheid en onderhoud.

## 2. Eén suite, maar bewust vier onafhankelijke onderdelen

PV Energy Suite is geen grote monolithische automation. De repository bundelt vier componenten die zelfstandig kunnen draaien. Dat is belangrijk: je kunt klein beginnen en later uitbreiden zonder dat je direct de hele architectuur nodig hebt.



De verdeling is scherp. De blueprint schakelt. De forecast voorspelt. De planner adviseert. Phase Guard bewaakt. Daardoor is duidelijk welk onderdeel verantwoordelijk is wanneer er iets gebeurt.

### Sterk ontwerpprincipe

De PV-blueprint is de enige schrijver naar de Shelly/PV-schakelaar. Forecast en planner lezen dezelfde drempel- en blokkeringshelpers om hun voorspellingen op dezelfde werkelijkheid te baseren, maar sturen de hardware niet zelf.

### 3. Welke gegevens komen de suite binnen?

De suite maakt brondata niet zelf. Hij combineert bestaande Home Assistant-entiteiten uit prijs-, zonne-, omvormer- en slimme-meterintegraties. De actuele README benadrukt dit nu heel duidelijk.

| Bron                                   | Gebruikt door                        | Verwachte vorm                         |
|----------------------------------------|--------------------------------------|----------------------------------------|
| Zonneplan kwartierprijzen              | Forecast + planner                   | forecast-attribuut met kwartierprijzen |
| Solcast vandaag/morgen                 | Forecast + planner                   | detailedForecast-attribuut             |
| Actueel PV-vermogen uit ECU/omvormer   | Forecast + planner                   | W                                      |
| P1 netimport en netexport              | Forecast + planner                   | kW                                     |
| Drempel + auto + handmatige blokkering | Forecast van curtailment + blueprint | Home Assistant-helpers                 |
| Drie fasebronnen                       | Phase Guard                          | bij voorkeur stroom in A               |

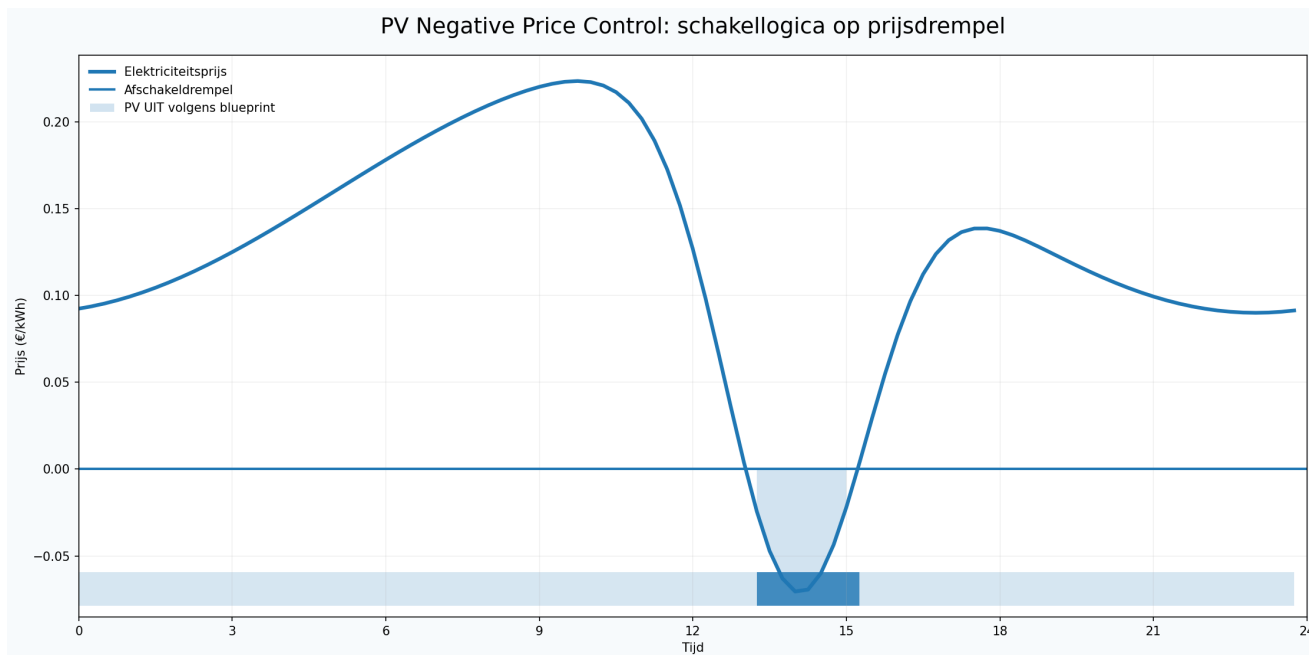
De eenheden zijn geen detail. De repo verwacht ECU-PV in watt, P1-import/export in kilowatt en Solcast detailedForecast in kilowatt. Verkeerde eenheden leveren anders zeer overtuigend ogende maar inhoudelijk foutieve resultaten op.

#### MQTT Discovery

De forecast, planner en Phase Guard publiceren hun resultaten via MQTT Discovery. Sensoren als `sensor.pv_*`, `sensor.pv_planner_*` en `sensor.phase_guard_*` hoef je dus niet zelf als helpers te maken.

## 4. PV Negative Price Control - het enige onderdeel dat echt schakelt

De blueprint vergelijkt de actuele elektriciteitsprijs met een instelbare drempel. Is de prijs kleiner dan of gelijk aan de drempel, dan kan de PV-installatie worden uitgeschakeld. Stijgt de prijs weer boven de drempel, dan kan de PV terug worden ingeschakeld.



De blueprint ondersteunt geïnverteerde schakellogica. Dat is relevant wanneer een contactor zo is bedraad dat switch aan juist betekent dat de PV uit staat. De README waarschuwt daarom terecht om `inverted_switch` altijd tegen de fysieke installatie te controleren.

Daarnaast zijn er helpers voor automatische regeling, handmatige blokkering, laatste schakelmoment en minimale schakeltijd. De blueprint bevat ook ondersteuning voor een optionele temperatuursensor.

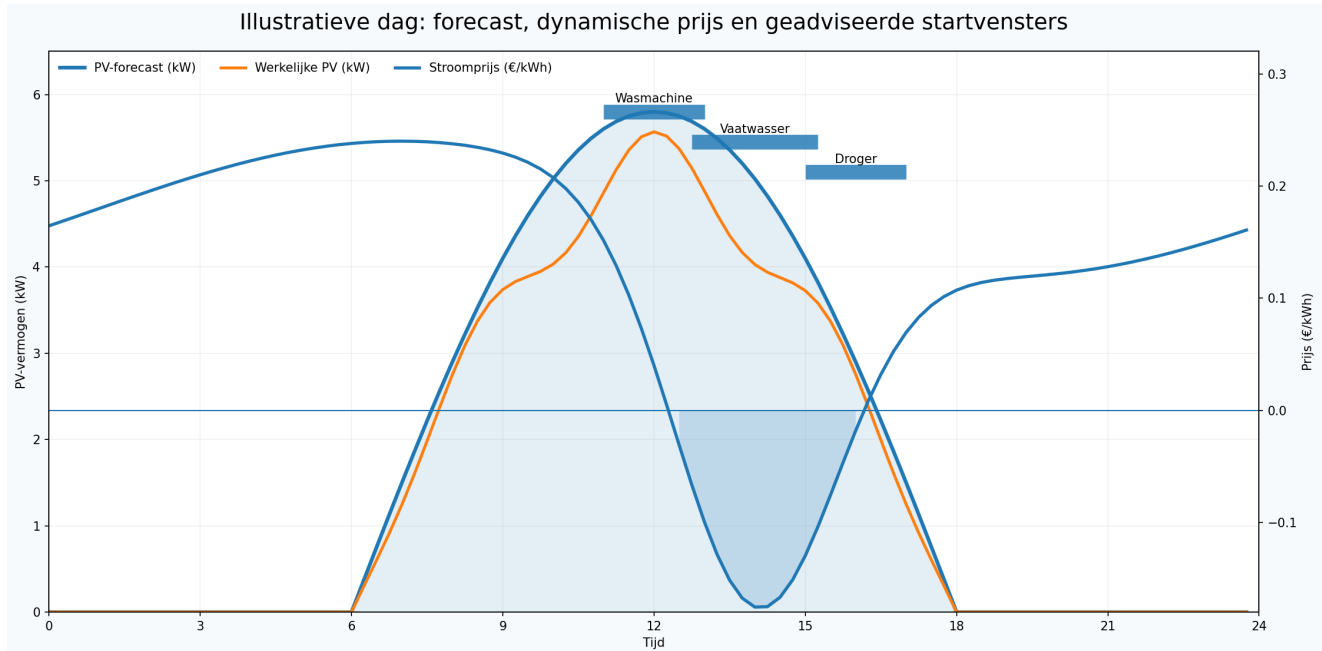
### Lees ook het hardware-artikel

Voor de elektrische kant - Shelly, contactor, fail-safe werking en fysieke bedrading - verwijs ik bewust naar het bestaande MechScript-artikel: [Zonnepanelen afschakelen met Shelly en contactor](#). Deze gids blijft aan de softwarekant.

## 5. PV Grid Forecast - van losse sensoren naar één energiebeeld

PV Grid Forecast combineert kwartierprijzen, Solcast, werkelijk PV-vermogen en P1-import/export. Het resultaat is niet alleen een zonneforecast, maar een tijdlijn waarop prijs, verwachte productie, werkelijk gedrag en het netbeeld naast elkaar kunnen worden beoordeeld.

De app kan daarnaast learning- en financiële functies gebruiken wanneer je de extra ECU- en Zonneplan-dagsensoren invult. Bestaande learning-JSON-bestanden moet je bij upgrades bewaren.



De bovenstaande grafiek is illustratief, maar laat de gedachte achter de suite goed zien: een hoge zonneproductie is niet automatisch het beste moment. De planner kijkt ook naar prijs, netimport en gemiste terugleverwaarde.

PV Grid Forecast kan bovendien de planner intern draaien. Dat is volgens de huidige README de aanbevolen route wanneer je de suite eerst wilt testen.

## 6. Self-Consumption Planner - slim verbruik plannen zonder apparaten over te nemen

De Self-Consumption Planner is het meest opvallende nieuwe onderdeel van de suite. Hij zoekt binnen een toegestaan tijdvenster naar een gunstige starttijd voor een apparaat. De standaardvoorbeelden zijn vaatwasser, wasmachine en droger, maar je kunt extra apparaten toevoegen.



Per apparaat kent de planner onder meer de programmaduur, het geschatte gemiddelde vermogen, het piekvermogen, de vroegste start en de uiterste eindtijd. Een apparaat doet alleen mee wanneer er daadwerkelijk vraag is.

### Heel bewust advisory-only

De planner start géén wasmachine, droger of vaatwasser. Ook niet wanneer Phase Guard en de forecast zijn geïnstalleerd. Hij geeft uitsluitend een advies. Dat is niet een ontbrekende functie, maar een bewuste veiligheids- en architectuurkeuze.

## 7. Hoe kiest de planner een goed tijdvenster?

In de code wordt elk kandidaatvenster doorgerekend. De score beloont eigen PV-gebruik en exportreductie en bestraft netimport, elektriciteitskosten en piekbelasting. Die onderdelen hebben configureerbare gewichten.

Concreet worden onder meer deze factoren meegenomen:

- hoeveel van het programmaverbruik rechtstreeks uit verwachte PV kan komen;
- hoeveel teruglevering daardoor wordt voorkomen;
- hoeveel energie alsnog uit het net moet worden geïmporteerd;
- wat die import kost én welke terugleveropbrengst je misloopt door zelf te verbruiken;
- hoe groot de geschatte netpiek tijdens het programma wordt.

De planner kan bovendien een `pv_safety_margin_pct` toepassen. Daardoor plant hij niet alsof de zonneforecast gegarandeerd exact wordt gehaald.

In de broncode staat het principe compact samengevat: meer self-consumption en exportreductie verhogen de score; grid import, kosten en piekbelasting verlagen hem. Dat maakt de planner duidelijk méér dan “kies het zonnigste uur”.

### Belangrijk verschil

De globale Phase Guard-headroom wordt niet als harde startbeveiliging gebruikt voor een apparaat waarvan de fase onbekend is. De repository waarschuwt daar expliciet voor.

## 8. Optionele vermogenssensor: de planner kan leren van echte cycli

Een apparaatprofiel kan optioneel een actuele vermogenssensor krijgen. De sensor moet uitsluitend dat apparaat meten en vermogen in W of kW leveren. De planner bemonstert die meting iedere minuut.

Na een afgeronde cyclus kan de planner het gemeten verbruiksprofiel gebruiken voor toekomstige adviezen. In de code wordt voor het gemiddelde vermogen de mediaan van de opgeslagen cyclusgemiddelden gebruikt; het geschatte piekvermogen wordt gebaseerd op de hoogste waargenomen piek in die cycli.

Een losse 0 W wanneer de machine uit staat wordt dus niet ineens het voorspelde programmaverbruik. De programmaduur blijft bewust handmatig, omdat verschillende programma's op hetzelfde apparaat sterk in duur kunnen verschillen.

De huidige README maakt dit onderscheid helder: meten helpt het vermogen realistischer maken, maar de gebruiker blijft verantwoordelijk voor de gekozen programmaduur en deadline.

### Praktische instelling

Als je 1,2 kWh meet over een programma van 2 uur, is 600 W een redelijke eerste schatting voor het gemiddelde vermogen. Meet piekvermogen apart wanneer dat relevant is. Gebruik daarna de echte vermogenssensor zodat de planner zijn model kan verfijnen.

## 9. Geïntegreerde planner of standalone? Kies precies één

De suite ondersteunt twee manieren om de planner te draaien. Ze publiceren dezelfde MQTT Discovery-ID's en topics. Daarom mogen ze nooit tegelijk actief zijn.

| Modus              | Bestanden                                              | Configuratie                                                | Forecastbasis                                                 |
|--------------------|--------------------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------|
| Forecast + planner | pv_grid_forecast.py +<br>pv_self_consumption.py        | pv_grid_forecast: met<br>planner_enabled: true              | kan de forecast/learning<br>van PV Grid Forecast<br>gebruiken |
| Standalone planner | pv_self_consumption.py +<br>pv_self_consumption_app.py | pv_self_consumption: uit<br>standalone_planner.yaml.example | leest bronnen zelf; ruwe<br>Solcast P50                       |
| Alleen forecast    | pv_grid_forecast.py                                    | planner_enabled: false                                      | geen planneroutput                                            |

### Als je net begint

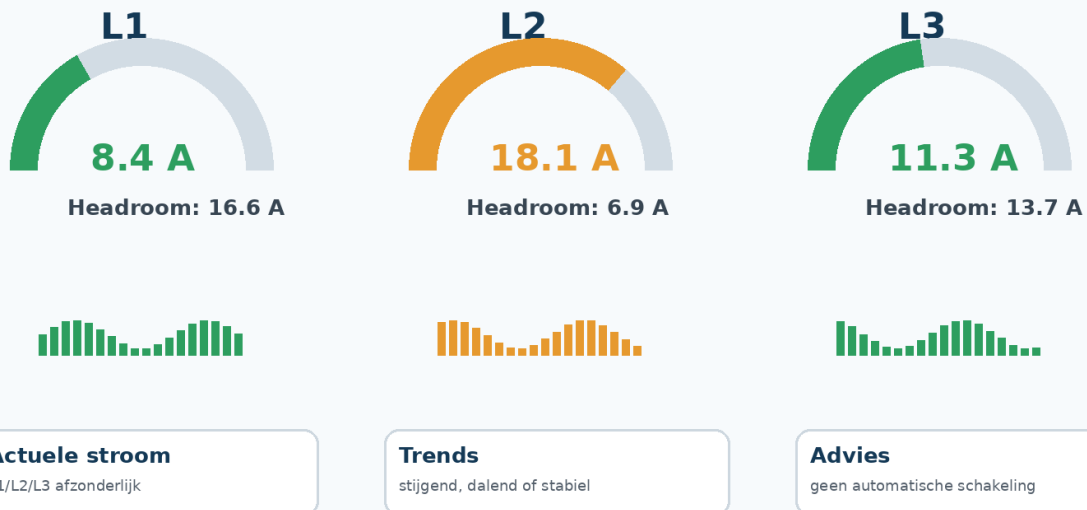
Gebruik de geïntegreerde planner. Dat is nu ook de expliciete quick-start in de README. Ga pas standalone draaien als je daar een concrete reden voor hebt.

## 10. Phase Peak Guard - de elektrische grens in beeld

Phase Peak Guard kijkt naar de drie fasen afzonderlijk. De app publiceert stroom, headroom en trends via MQTT Discovery. Hij schakelt niets uit; de output is bedoeld voor dashboards en menselijke besluitvorming.

### Phase Peak Guard: hoeveel ruimte heeft elke fase nog?

Illustratief voorbeeld voor een 3×25 A-aansluiting. De app meet, analyseert en adviseert - hij schakelt niets.



Dat is belangrijk bij 3×25 A: een totaal vermogen kan er normaal uitzien terwijl één fase veel zwaarder wordt belast dan de andere twee.

#### Lees ook het aparte artikel

De volledige werking, trendanalyse en interpretatie van Phase Peak Guard staat al in je bestaande MechScript-artikel: [Phase Peak Guard - Home Assistant](#). In deze suite-gids houden we de uitleg daarom bewust compact.

# 11. Installeren - de huidige quick-start uit de README

## Installatie in vijf controleerbare stappen

De README benadrukt nu expliciet: werk vanuit je bestaande apps.yaml en kies één planner-modus.

### 1 Bestanden

Kopieer alleen de apps die je echt gebruikt.

### 2 apps.yaml

Neem alleen het juiste voorbeeldblok over; overschrijf je bestaande bestand niet.

### 3 Bronentiteiten

Vervang REPLACE\_ME en controleer eenheden en entity-ID's.

### 4 AppDaemon + MQTT

Herstart AppDaemon en controleer logs en MQTT Discovery-entiteiten.

### 5 Dashboards

Voeg optionele Lovelace-views pas toe als de data klopt.

De aanbevolen eerste test is Forecast + geïntegreerde planner. Dat gaat als volgt:

- kopieer apps/pv\_grid\_forecast.py en apps/pv\_self\_consumption.py naar de map waarin jouw AppDaemon apps.yaml staat;
- open je bestaande apps.yaml en neem alleen het blok pv\_grid\_forecast: uit het voorbeeld over;
- zet planner\_enabled: true en voeg géén apart pv\_self\_consumption:-blok toe;
- vervang alle gebruikte voorbeeldentiteiten en REPLACE\_ME-waarden door je eigen entity-ID's;
- herstart AppDaemon en controleer eerst de log, daarna de MQTT Discovery-entiteiten.

### Niet overschrijven

De huidige README benadrukt terecht dat apps.yaml.example een voorbeeld is. Overschrijf nooit je bestaande AppDaemon-configuratie met het voorbeeldbestand.

## 12. De kern van apps.yaml - wat moet je echt begrijpen?

Onderstaand fragment laat alleen de architectuur zien. De echte repo bevat meer velden. Gebruik altijd de actuele `examples/apps.yaml.example` als bron en vervang je eigen entity-ID's.

### Vereenvoudigd voorbeeld - niet blind kopiëren

```
pv_grid_forecast:
  module: pv_grid_forecast
  class: PVGridForecast
  planner_enabled: true
  price_quarter_entity: sensor.zonneplan...
  solcast_today_entity: sensor.solcast..._today
  solcast_tomorrow_entity: sensor.solcast..._tomorrow
  ecu_power_entity: sensor.jouw_pv_vermogen
  grid_import_entity: sensor.power_consumption
  grid_export_entity: sensor.power_production
  threshold_entity: input_number.pv_negatieve_prijs_drempel
  automatic_entity: input_boolean.pv_automatisch_afschakelen
  manual_block_entity: input_boolean.pv_handmatig_geblokkeerd
  advisory_only: true
  automation_enabled: false
```

De belangrijke boodschap zit in de rollen: brontentiteiten komen uit Home Assistant; de app gebruikt ze; de resultaten verschijnen weer als MQTT Discovery-entiteiten. De planner staat in het publieke voorbeeld expliciet op `advisory_only: true` en `automation_enabled: false`.

Voor Phase Guard voeg je een apart `phase_peak_guard`-blok toe met drie fasebronnen en een eigen JSON-opslagpad.

## 13. Apparaten instellen: optie A of optie B

De vernieuwde README splitst dit nu netjes op in twee routes.

### Optie A - snel testen met vaste YAML-waarden

Verwijder in één apparaatblok alle velden die eindigen op `_entity`. Vul daarna duur, gemiddeld vermogen, piekvermogen, vroegste start en uiterste eindtijd rechtstreeks in YAML in. Zet `enabled: true`. Let op: bij vaste waarden blijft het apparaat iedere dag in het advies staan tot je `enabled` weer uitzet.

### Optie B - Home Assistant-helpers en dashboard

Activeer Home Assistant-packages en plaats `planner_helpers.yaml.example` als `packages/pv_planner_helpers.yaml`. Per apparaat krijg je zes basishelpers plus een optionele teksthelper voor de vermogenssensor.

| Helper                                        | Functie                                    |
|-----------------------------------------------|--------------------------------------------|
| <code>input_boolean</code>                    | apparaat moet gepland worden               |
| <code>input_number</code> duur                | programmaduur in minuten                   |
| <code>input_number</code> gemiddeld vermogen  | eerste schatting / handmatige waarde       |
| <code>input_number</code> piekvermogen        | verwachte piek                             |
| <code>input_datetime</code> vroegste start    | begin van toegestaan venster               |
| <code>input_datetime</code> uiterste eindtijd | deadline                                   |
| <code>input_text</code> vermogenssensor       | optionele sensor.*-entity voor live meting |

## 14. De vier meegeleverde dashboardviews

De repository bevat vier Lovelace-views. Het zijn losse views voor de ruwe YAML-editor, geen complete dashboards. Dat onderscheid is belangrijk.

| View              | Doel                                           | Extra frontendkaarten                                        |
|-------------------|------------------------------------------------|--------------------------------------------------------------|
| PV Grid Forecast  | forecast, netstatus, verbruiksadvies, tijdlijn | Mushroom, Layout Card, ApexCharts, Fold Entity Row, card-mod |
| PV-overzicht      | eigen PV/P1/prijs/Shelly-overzicht             | Mushroom, Solar Forecast Card, Restriction Card, card-mod    |
| Phase Guard       | L1/L2/L3, headroom en trends                   | geen extra kaarten                                           |
| Apparaten plannen | helpers en planneradvies per apparaat          | geen extra kaarten                                           |

De forecast-view gebruikt vooral output die de app zelf via MQTT maakt. Het PV-overzicht is anders: veel van de kaarten verwachten jouw eigen PV-, P1-, prijs- en Shelly-entiteiten. Zoek daarom vóór gebruik naar `replace_me` en `REPLACE_WITH_`.

### Goede volgorde

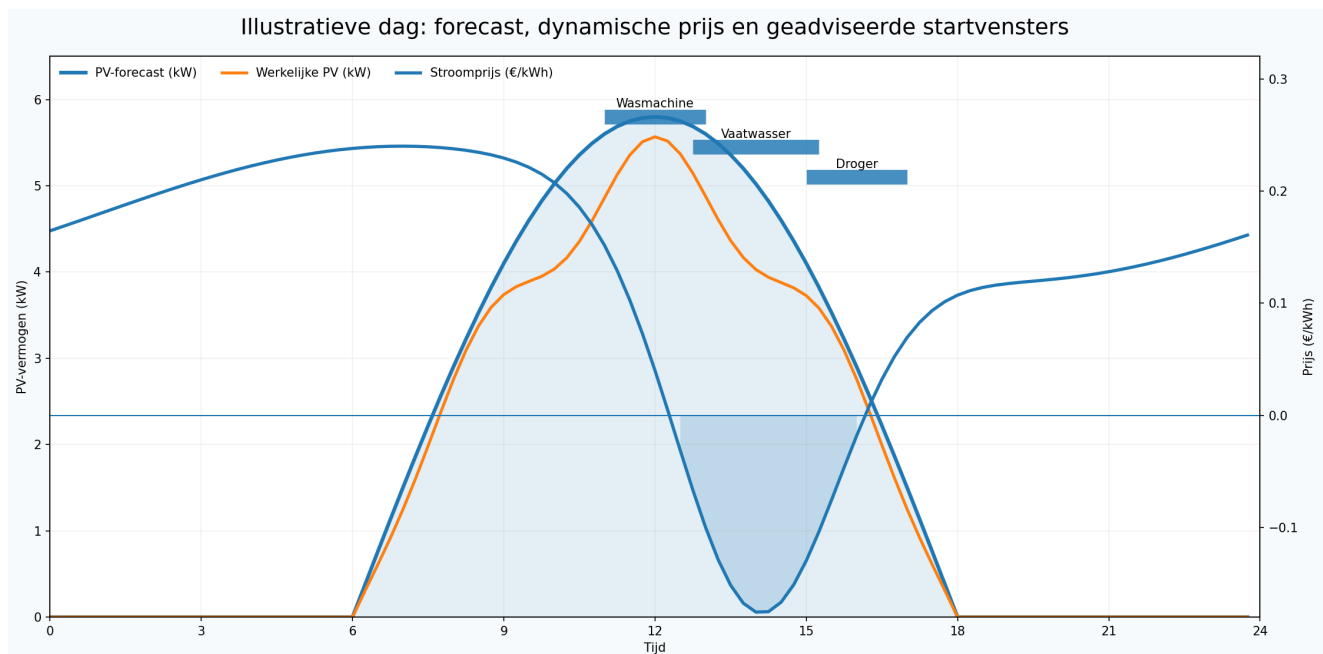
Zorg eerst dat de apps en MQTT-entiteiten correct werken. Importeer pas daarna dashboards. Een fraai dashboard kan een fout datamodel niet repareren.

## 15. Hoe ziet een complete dag er in de praktijk uit?

Stel: Solcast voorspelt rond de middag veel productie, terwijl de kwartierprijzen teruglopen en een deel van de middag negatief wordt. Tegelijk wil je de wasmachine en vaatwasser vandaag laten draaien.

Dan kan de suite ongeveer zo worden gebruikt:

- de forecast bouwt de tijdlijn uit prijs, Solcast, actuele PV en P1;
- de planner beoordeelt voor elk gevraagd apparaat kandidaatvensters binnen de ingestelde deadlines;
- je ziet bijvoorbeeld dat 11:00-13:00 gunstig is voor de wasmachine en 12:45-15:15 voor de vaatwasser;
- de blueprint beoordeelt onafhankelijk of de actuele prijs onder de ingestelde PV-drempel komt en kan dan de PV-schakelaar bedienen;
- Phase Guard blijft tegelijk de drie fasen volgen zodat je op het dashboard ziet hoeveel elektrische ruimte er nog is.



Dit voorbeeld is illustratief. De suite voorspelt en adviseert op basis van jouw echte brondata; hij garandeert geen productie of prijs en start de huishoudelijke apparaten niet zelf.

## 16. Wat automatiseert PV Energy Suite bewust níét?

Dit is misschien wel het belangrijkste hoofdstuk voor vertrouwen in het systeem.

- De Self-Consumption Planner start geen apparaten.
- Phase Peak Guard schakelt geen verbruikers uit.
- PV Grid Forecast bedient de PV-schakelaar niet.
- Alleen de PV Negative Price Control-blueprint mag de PV-Shelly/schakelaar schrijven.
- De suite maakt jouw bronentiteiten niet aan; je moet de juiste integraties zelf hebben.
- De globale fase-headroom is geen harde startbeveiliging voor een apparaat op een onbekende fase.

Deze grenzen zijn juist gezond. Ze voorkomen dat een forecastmodule ineens hardware gaat bedienen of dat een planner zonder kennis van de fysieke fase een zware belasting als “veilig” beschouwt.

### Voor automatische apparaatsturing

Wil je later apparaten automatisch starten, bouw dat als aparte automation met expliciete voorwaarden en beveiligingen. Gebruik het planneradvies als input, niet als impliciete toestemming.

## 17. Ingebruikname, troubleshooting en upgrades

Na installatie controleer je eerst de AppDaemon-log en pas daarna de dashboards. Kijk vervolgens of de MQTT-devices online zijn en of de bronentiteiten de juiste eenheden leveren.

Typische problemen:

- `insufficient_data`: één of meer benodigde bronnen ontbreken of leveren geen bruikbare data; de planner wist dan het actuele advies;
- geen planner-entiteiten: geïntegreerde planner niet actief, MQTT niet goed of AppDaemon heeft een fout;
- vreemde planning: controleer duur, gemiddeld vermogen, piekvermogen, tijdvenster en bron-eenheden;
- dubbele/onrustige MQTT-entiteiten: controleer of geïntegreerde en standalone planner niet tegelijk draaien;
- PV schakelt verkeerd om: controleer `inverted_switch` tegen de fysieke relaislogica.

Bij upgrades geldt één belangrijke regel: bewaar bestaande JSON-learningbestanden. Elke zelfstandige app hoort bovendien een eigen opslagpad te hebben; deel JSON-bestanden niet tussen apps.

### Bronnen en links

[GitHub - dkwolf1/pv-energy-suite](#)

[MechScript - Phase Peak Guard](#)

[MechScript - Zonnepanelen afschakelen met Shelly en contactor](#)

## 18. Conclusie

PV Energy Suite is vooral interessant omdat het verschillende energievraagstukken niet op één hoop gooit. Forecast, verbruiksplanning, fasebewaking en PV-afschakeling hebben elk hun eigen verantwoordelijkheid. Daardoor kun je de suite klein inzetten en gecontroleerd uitbreiden.

De vernieuwde README maakt de instap merkbaar beter: eerst Forecast + geïntegreerde planner testen, één configuratiemethode kiezen voor apparaten, bronentiteiten expliciet controleren en daarna pas dashboards en extra onderdelen toevoegen. Dat is precies de volgorde die ik ook voor een nieuwe installatie zou adviseren.

### Kort samengevat

Met de suite maak je van losse energiegegevens een bruikbaar beslissysteem: voorspellen wat er komt, adviseren wanneer je verbruikt, bewaken wat je aansluiting aankan en alleen daar automatisch schakelen waar dat expliciet is ontworpen.